

Basic predatory prey models

Anthony Davidson

16 August 2017

The first models of interactions of consumer-prey model

Model 1

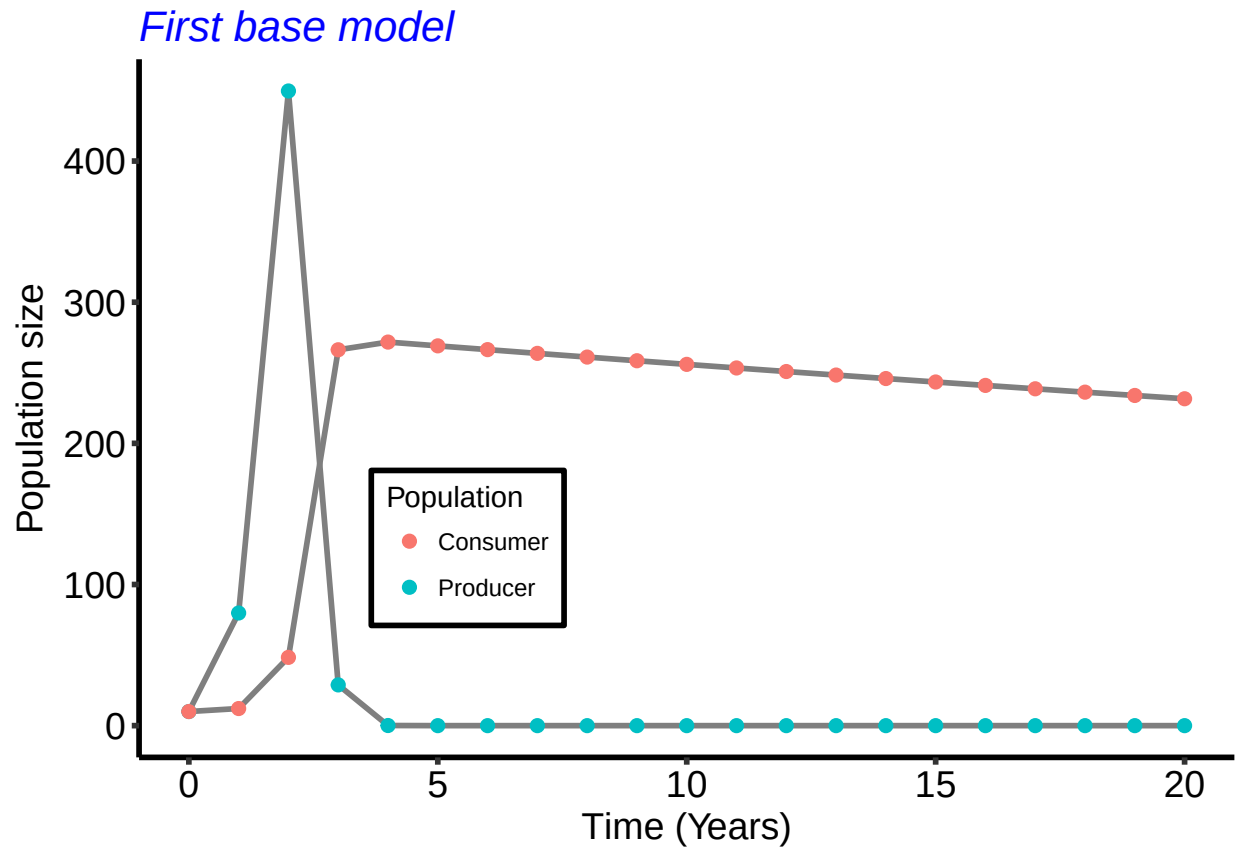
Here is a predator - prey model (Classic Lotka-Volterra two species model):

This is the differential equation for the prey population in the next time step.

$$\frac{dP}{dt} = r_g \times P \times \left(1 - \frac{P}{K}\right) - r_I \times P \times C$$

$$\frac{dC}{dt} = k_{AE} \times r_I \times P \times C - r_M \times C$$

```
Tmod0D <- function(Time, State, Pars) {  
  
  with(as.list(c(State, Pars)), {  
    IngestC <- rI * P * C  
    GrowthR <- rG * P * (1 - P/K)  
    MortC <- rM * C  
  
    dR <- GrowthR - IngestC  
    dC <- kAE * IngestC - MortC  
  
    return(list(c(dR, dC)))  
  })  
  
} #function ended  
  
# input estimates  
pars <- c(rI = 0.03, #ingestion rate of consumer  
          rG = 2.4,  #growth rate of prey  
          rM = 0.01, #consumer mortality  
          kAE = 0.2, #assimilation efficiency  
          K = 30000) #carrying capacity  
  
yini <- c(P = 10, C = 10) #initial values  
times <- seq(0, 20, by = 1) #length of projection  
  
# analysis  
out <- ode(func = Tmod0D, y = yini, parms = pars, time = times)  
  
# dataframe of output/results  
out1 <- data.frame(out)  
out1 <- gather(out1, Population, N, P:C)
```



And here is the predator (consumer) dynamics. It simply implies that in the next time step will be a result of the the previous one in this way.

And this is similar to the consumer-resource model for the first equation (estimating the Resource equation) but the second half of the equation is different because the consumers (C) increase in proportion to there growth rate..... [explain this better]

$$\frac{d\mathbf{R}}{dt} = r_g \times R \times \left(1 - \frac{R}{K}\right) - r_I \times R \times C$$

$$\frac{d\mathbf{C}}{dt} = r_I \times R \times C \times k_{AE} - C \times r_M \times \frac{C}{R}$$

$$\frac{d\mathbf{R}}{dt} = growth\mathbf{R} - ingest\mathbf{C}$$

$$\frac{d\mathbf{C}}{dt} = (k_{AE} \times ingest\mathbf{C}) - mort\mathbf{C}$$

Which equates too:

```
LVmod0D <- function(Time, State, Pars) {

  with(as.list(c(State, Pars)), {
    IngestC <- rI * R * C
    GrowthR <- rG * R * (1 - R/K)
    MortC <- C * rM * (C / R)
```

```

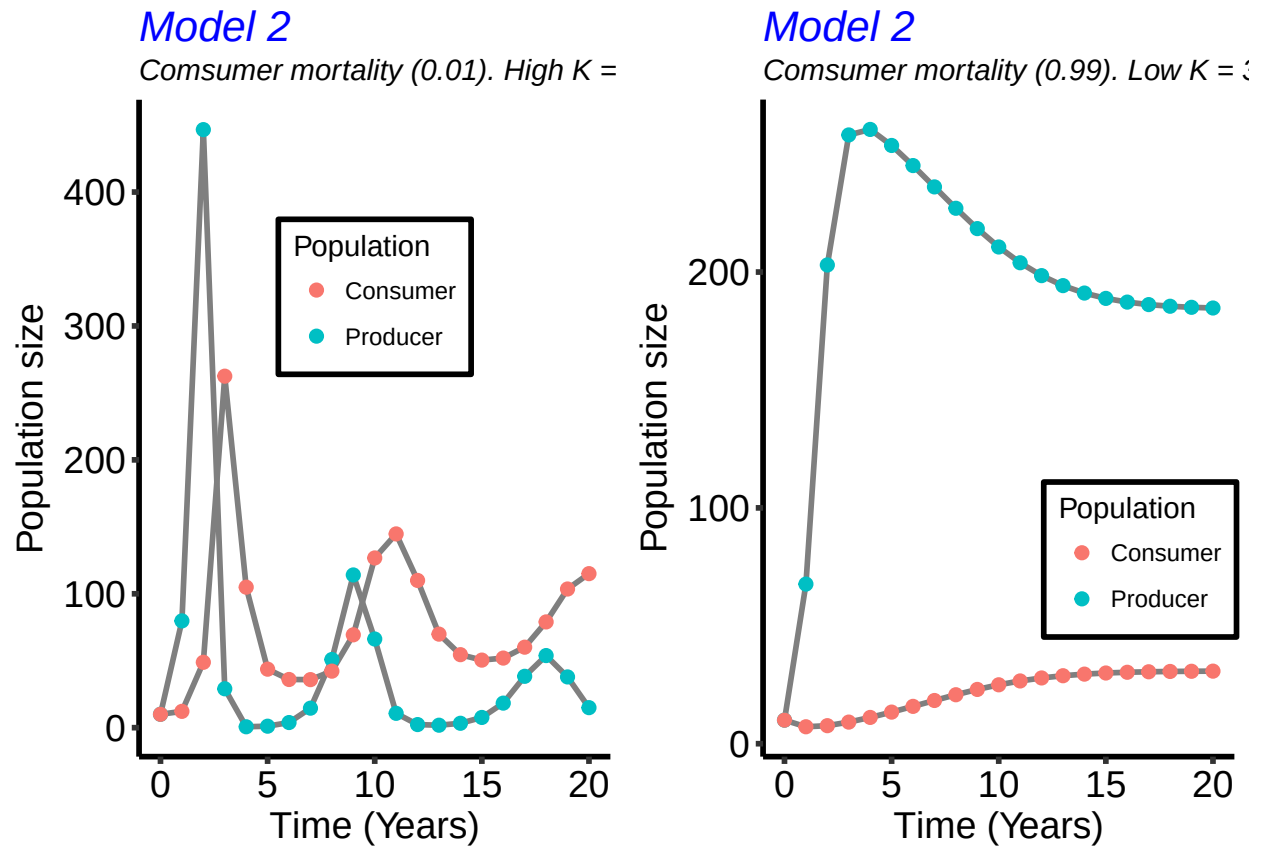
dR <- GrowthR - IngestC
dC <- IngestC * kAE - MortC

return(list(c(dR, dC)))
})

} #function ended

```

Then use a model that looks something like below:



Below is the working for the estimate linked with likelihood

The data

```

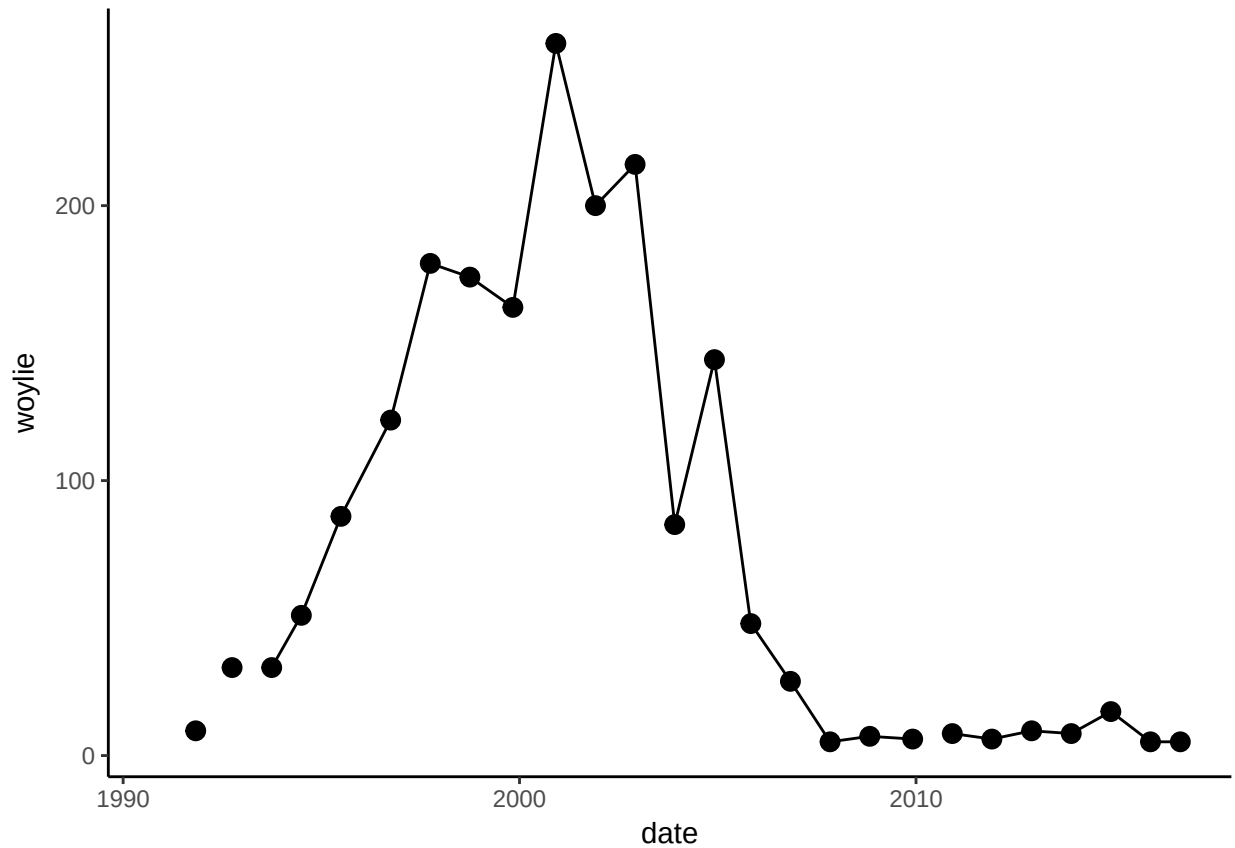
dat <- read.csv("woylie_ktba.csv")

dat <- dat %>%
  mutate(date = paste("01-", date, sep = "-"),
         date = dmy(date))

ggplot(dat, aes(y = woylie, x = date)) +
  geom_point(size = 3) +
  geom_line() +
  theme_classic()

```

```
## Warning: Removed 6 rows containing missing values (geom_point).
## Warning: Removed 1 rows containing missing values (geom_path).
```



Parameters set

```
#####
# fit directly to N data allowing for time differences (in years) between measurements
year <- decimal_date(dat$date)
n_animal <- dat$woylie
# remove missing values
keep <- is.na(n_animal) == F
year <- year[keep]
n_animal <- n_animal[keep]
yr <- year - min(year)
```

Consumer - resource differential equation model - C and R are consumer and resource numbers - rI is the ingestion rate of the consumer - rG is the growth rate of the resource - kAE is the assimilation efficiency - rM is the consumers mortality rate - Here mortality increases as a function of per capita resource availability

```
LVmod0D <- function(Time, State, Pars) {
  with(as.list(c(State, Pars)), {
    IngestC <- rI * R * C
    GrowthR <- rG * R * (1 - R/K)
    MortC <- C * rM * (C / R)
    dR <- GrowthR - IngestC
```

```

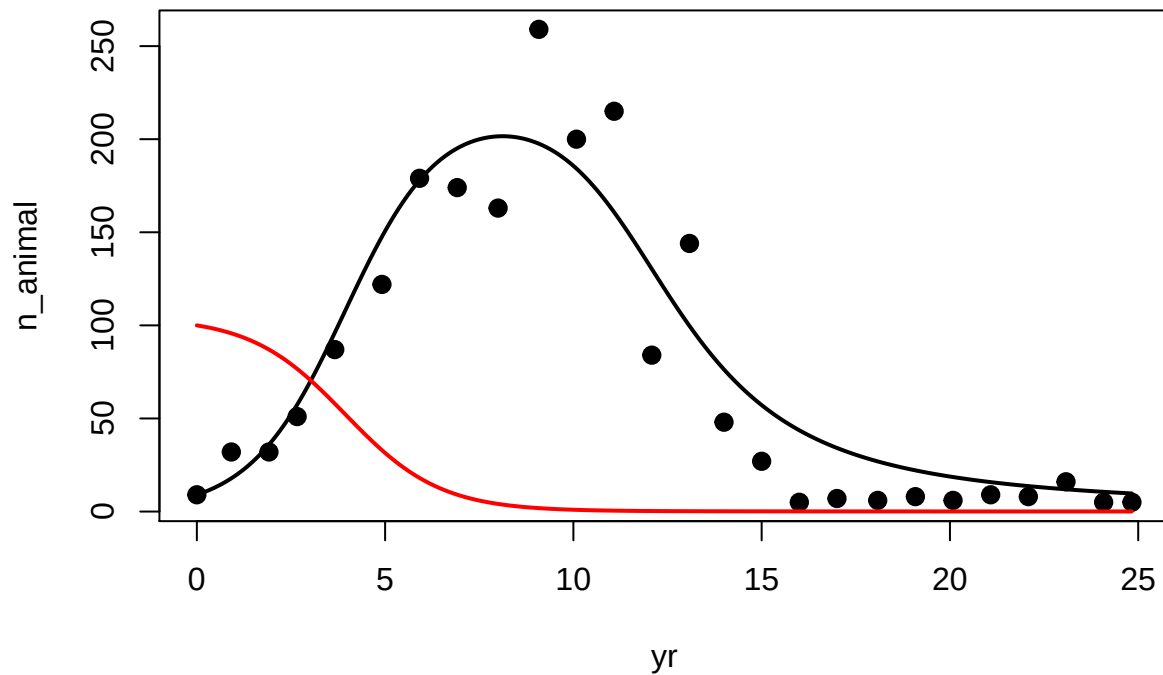
dC <- IngestC * kAE - MortC
return(list(c(dR, dC)))
})
}
# likelihood function
lik.fun <- function(param, y, yr) {
  rI <- exp(param[1])
  rG <- exp(param[2])
  rM <- exp(param[3])
  kAE <- exp(param[4])
  sigma <- exp(param[5])
  K <- 100
  pars <- c(rI = rI, rG = rG, rM = rM, K = K, kAE = kAE)
  yini <- c(R = 100, C = n_animal[1])
  out <- as.data.frame(ode(func = LVmod0D, y = yini, parms = pars, times = yr))
  mu <- out$C
  logl <- sum(dnorm(y, mean = mu, sd = sigma, log = T))
  return(-logl)
}

m1 <- optim(par = c(-6.7, -1.9, -0.9, 1.9, 3.2), fn=lik.fun, y = n_animal, yr = yr)
m1

## $par
## [1] -5.6052642 -17.4298573 -7.5423944 0.7288463 3.2942809
##
## $value
## [1] 122.5335
##
## $counts
## function gradient
##      395      NA
##
## $convergence
## [1] 0
##
## $message
## NULL

rI <- exp(m1$par[1])
rG <- exp(m1$par[2])
rM <- exp(m1$par[3])
kAE <- exp(m1$par[4])
K <- 100
xx <- seq(min(yr), max(yr), 0.01)
pars <- c(rI = rI, rG = rG, rM = rM, kAE = kAE, K = K)
yini <- c(R = 100, C = n_animal[1])
out <- as.data.frame(ode(func = LVmod0D, y = yini, parms = pars, times = xx))
plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$C ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red")

```



```
# rm is given by growth rate when resources are at maximum: R = 50
(rI * kAE * K) - rM
```

```
## [1] 0.7618975
```

Include two sets of data now

Parameters set

```
#####
# fit directly to N data allowing for time differences (in years) between measurements

#data used
dat <- read.csv("woylie_ktba.csv")
dat <- dat %>%
mutate(date = paste("01-", date, sep = ""),
date = dmy(date),
seed = rep(NA, length(dat$woylie)))

#create seed data
for(i in 1:length(dat$woylie)){
  dat$seed[i] <- ifelse(dat$woylie[i] > 0, rnorm(1,dat$woylie[i],20),NA)
} #finish loop

#remove negatives
```

```

for(i in 1:length(dat$seed)){
  dat$seed[i] <- ifelse(dat$seed[i] < 0, 0, dat$seed[i])
} #finish loop
#Animals
n_animal <- dat$woylie
keep <- is.na(n_animal) == F # remove missing values
n_animal <- n_animal[keep]

#years
year <- decimal_date(dat$date)
year <- year[keep]
yr <- year - min(year)

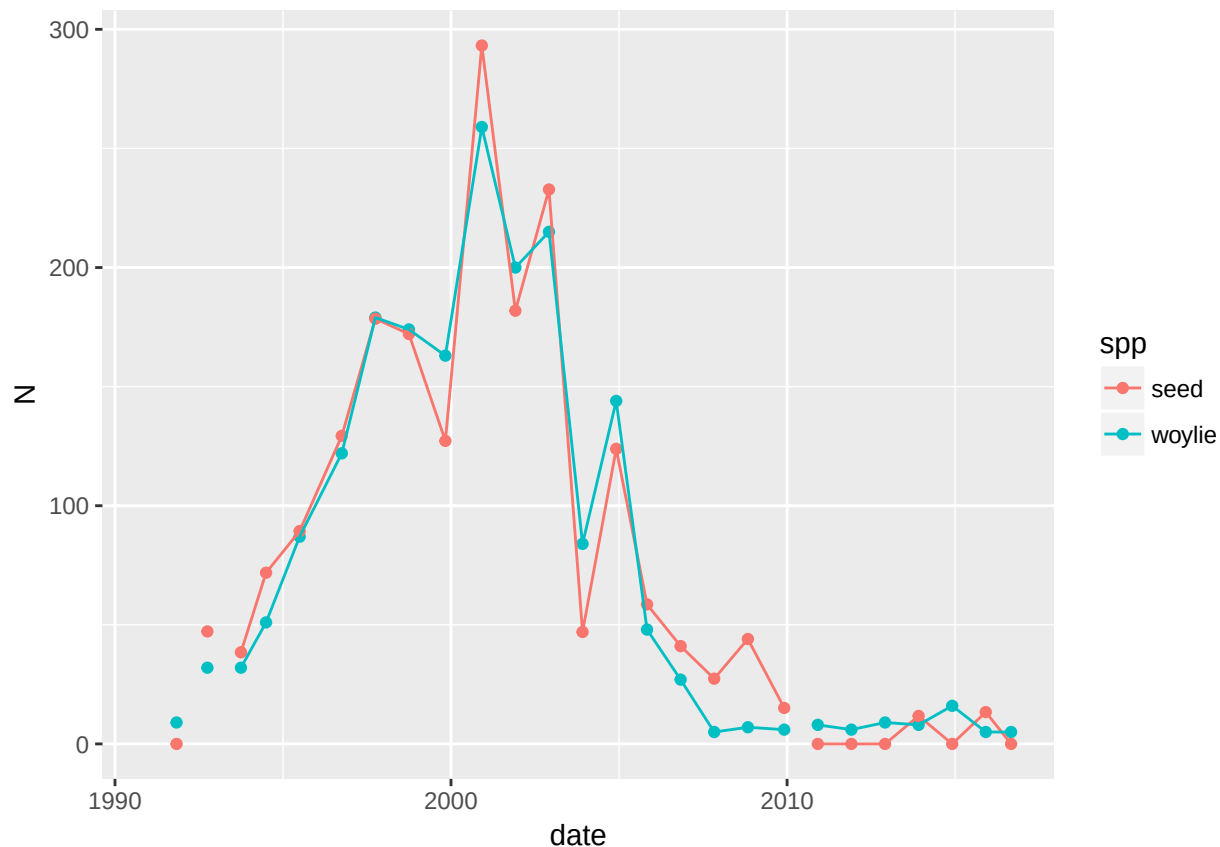
#Food
#Animals
n_food <- dat$seed
keep <- is.na(n_food) == F # remove missing values
n_food <- n_food[keep]
glimpse(dat)

## Observations: 32
## Variables: 3
## $ date    <date> 1990-12-01, 1991-11-01, 1992-02-01, 1992-05-01, 1992-0...
## $ woylie  <int> NA, 9, NA, NA, NA, 32, NA, 32, 51, 87, 122, 179, 174, 1...
## $ seed    <dbl> NA, 0.00000, NA, NA, NA, 47.22093, NA, 38.48441, 71.872...
dat1 <- gather(dat, spp, N, woylie:seed) %>%
  mutate(spp = factor(spp),
         N = as.numeric(N))

ggplot(dat1, aes(y = N, x = date, color = spp)) +
  geom_point() +
  geom_line(aes(group = spp))

## Warning: Removed 12 rows containing missing values (geom_point).
## Warning: Removed 2 rows containing missing values (geom_path).

```



#can't work out why it will not draw lines for first two points...
`table(dat1$spp, dat1$date)`

```
##
##      1990-12-01 1991-11-01 1992-02-01 1992-05-01 1992-08-01 1992-10-01
## seed          1          1          1          1          1          1
## woylie         1          1          1          1          1          1
##
##      1993-02-01 1993-10-01 1994-07-01 1995-07-01 1996-10-01 1997-10-01
## seed          1          1          1          1          1          1
## woylie         1          1          1          1          1          1
##
##      1998-10-01 1999-11-01 2000-12-01 2001-12-01 2002-12-01 2003-12-01
## seed          1          1          1          1          1          1
## woylie         1          1          1          1          1          1
##
##      2004-12-01 2005-11-01 2006-11-01 2007-11-01 2008-11-01 2009-12-01
## seed          1          1          1          1          1          1
## woylie         1          1          1          1          1          1
##
##      2010-06-01 2010-12-01 2011-12-01 2012-12-01 2013-12-01 2014-12-01
## seed          1          1          1          1          1          1
## woylie         1          1          1          1          1          1
##
##      2015-12-01 2016-09-01
## seed          1          1
```

```
##   woylie           1           1
str(dat1)

## 'data.frame':   64 obs. of  3 variables:
## $ date: Date, format: "1990-12-01" "1991-11-01" ...
## $ spp : Factor w/ 2 levels "seed","woylie": 2 2 2 2 2 2 2 2 2 ...
## $ N   : num  NA 9 NA NA NA 32 NA 32 51 87 ...

dat$seed

## [1]      NA  0.00000      NA      NA      NA  47.22093      NA
## [8] 38.48441 71.87202 89.34246 129.33996 178.49618 172.10011 127.21637
## [15] 293.20030 181.87166 232.75577 47.00352 123.87112 58.58272 41.05705
## [22] 27.38377 44.04618 15.10718      NA  0.00000  0.00000  0.00000
## [29] 11.75314  0.00000 13.37073  0.00000

dat$woylie

## [1]  NA   9  NA  NA  NA  32  NA  32  51  87 122 179 174 163 259 200 215
## [18] 84 144 48 27  5  7  6  NA  8  6  9  8 16  5  5

head(dat)

##      date woylie      seed
## 1 1990-12-01      NA      NA
## 2 1991-11-01      9 0.00000
## 3 1992-02-01      NA      NA
## 4 1992-05-01      NA      NA
## 5 1992-08-01      NA      NA
## 6 1992-10-01     32 47.22093
```

Set up the base equation for Mice and Beech seed

```
baseMice1<- function(Time, State, Pars) {

  with(as.list(c(State, Pars)), {

    GrowthM <- c1 * R
    denI <- mu1 * M
    denD <- mu2 * M * M

    dM <- GrowthM - denI - denD

  return(list(c(dM)))
})
}

# likelihood function
lik.fun <- function(param, y, yr) {
  c1 <- exp(param[1])
  mu1 <- exp(param[2])
  mu2 <- exp(param[3])

  pars <- c(c1 = c1, mu1 = mu1, mu2 = mu2)
```

```

yini <- c(R = 100, M = n_animal[1])
out <- as.data.frame(ode(func = baseMice1, y = yini, parms = pars, times = yr))
mu <- out$M
logl <- sum(dnorm(y, mean = mu, sd = sigma, log = T))
return(-logl)
}

baseMice2<- function(Time, State, Pars) {

  with(as.list(c(State, Pars)), {

    growthM <- M * max.lam * (1- M/K)
    conM <- c1 * R * M #simplifies because alpha (demographic effiecney = one)
    denI <- mu1 * M
    denD <- mu2 * M * M
    avalF <- h * R
    seedF <- Fy * R/k

    dM <- growthM - conM - denI - denD
    dF <- seedF - avalF - conM

    ###this bit was removed as explained in results because model was too sensitive to input parameters

    return(list(c(dM,dF)))
  }) #end of with statement

  ##is this where seed goes??

} #end

xx <- seq(min(yr), max(yr), 0.01)
pars <- c(c1 = 4.41,
          mu1 = -1.31,
          mu2 = 0.77,
          h = 9.18,
          Fy = 1.8,
          max.lam = 4.50,
          K = 149,
          k = 25000)

yini <- c(R = 100, M = 100)

out <- as.data.frame(ode(func = baseMice2,
                        y = yini,
                        parms = pars,
                        times = xx))

## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine, T + H = T on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 1.14725e-17
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine, T + H = T on the next step

```

```

##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 1.14725e-17
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 1.14725e-17
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 1.14725e-17
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 9.43929e-18
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 9.43929e-18
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 9.43929e-18
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 9.43929e-18
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 7.76641e-18
##
## DLSODA- Warning..Internal T (=R1) and H (=R2) are
##      such that in the machine,  $T + H = T$  on the next step
##      (H = step size). Solver will continue anyway.
## In above message, R1 = 0.210727, R2 = 7.76641e-18
##
## DLSODA- Above warning has been issued I1 times.
##      It will not be issued again for this problem.
## In above message, I1 = 10
##
## DLSODA- At current T (=R1), MXSTEP (=I1) steps
##      taken on this call before reaching TOUT
## In above message, I1 = 5000
##
## In above message, R1 = 0.210727
##

```

```
## Warning in lsoda(y, times, func, parms, ...): an excessive amount of work
## (> maxsteps ) was done, but integration was not successful - increase
## maxsteps
```

```
## Warning in lsoda(y, times, func, parms, ...): Returning early. Results are
## accurate, as far as they go
```

```
#sorting data to bind
dat3 <- dat1
levels(dat3$spp) <- c("R", "M")
dat3$date <- decimal_date(dat3$date)
dat3$date = round(dat3$date - min(dat3$date), digits = 2)
dat3 <- rename(dat3, time = date, true.N = N)
glimpse(dat3)
```

```
## Observations: 64
## Variables: 3
## $ time    <dbl> 0.00, 0.92, 1.17, 1.42, 1.67, 1.83, 2.17, 2.83, 3.58, 4...
## $ spp      <fctr> M, M, M, M, M, M, M, M, M, M, M, M, M, M, M, M, ...
## $ true.N    <dbl> NA, 9, NA, NA, NA, 32, NA, 32, 51, 87, 122, 179, 174, 1...
```

```
#short to long format for ggplot
out1 <- gather(out, spp, N, R:M) %>%
  droplevels()
```

```
glimpse(out1)
```

```
## Observations: 46
## Variables: 3
## $ time <dbl> 0.0000000, 0.0100000, 0.0200000, 0.0300000, 0.0400000, 0....
## $ spp <chr> "R", "R", "R", "R", "R", "R", "R", "R", "R", "R", "R", "R...
## $ N    <dbl> 1.000000e+02, 1.194471e+01, 4.230093e+00, 1.611690e+00, 4...
```

```
#bind two datasets
dat4 <- left_join(dat3,out1)
```

```
## Joining, by = c("time", "spp")
```

```
## Warning in left_join_impl(x, y, by$x, by$y, suffix$x, suffix$y): joining
## character vector and factor, coercing into character vector
```

```
glimpse(dat4)
```

```
## Observations: 64
## Variables: 4
## $ time    <dbl> 0.00, 0.92, 1.17, 1.42, 1.67, 1.83, 2.17, 2.83, 3.58, 4...
## $ spp      <chr> "M", "M", "M", "M", "M", "M", "M", "M", "M", "M", "M", ...
## $ true.N    <dbl> NA, 9, NA, NA, NA, 32, NA, 32, 51, 87, 122, 179, 174, 1...
## $ N         <dbl> 100, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA...
```

```
#plot them finally
ggplot(dat4, aes(y = N, x = time, group = spp, col = spp, shape = spp)) +
  geom_line(size = 1.2) +
  geom_point(aes(x = time, y = true.N), size = 3) +
#facet_wrap(~ pop) +
  ylab("Population size (N)") +
  xlab("Time (T)") +
  theme_bw() +
  scale_shape_discrete(labels = c("Mice (M)", "Seed (R)")) +
```

```

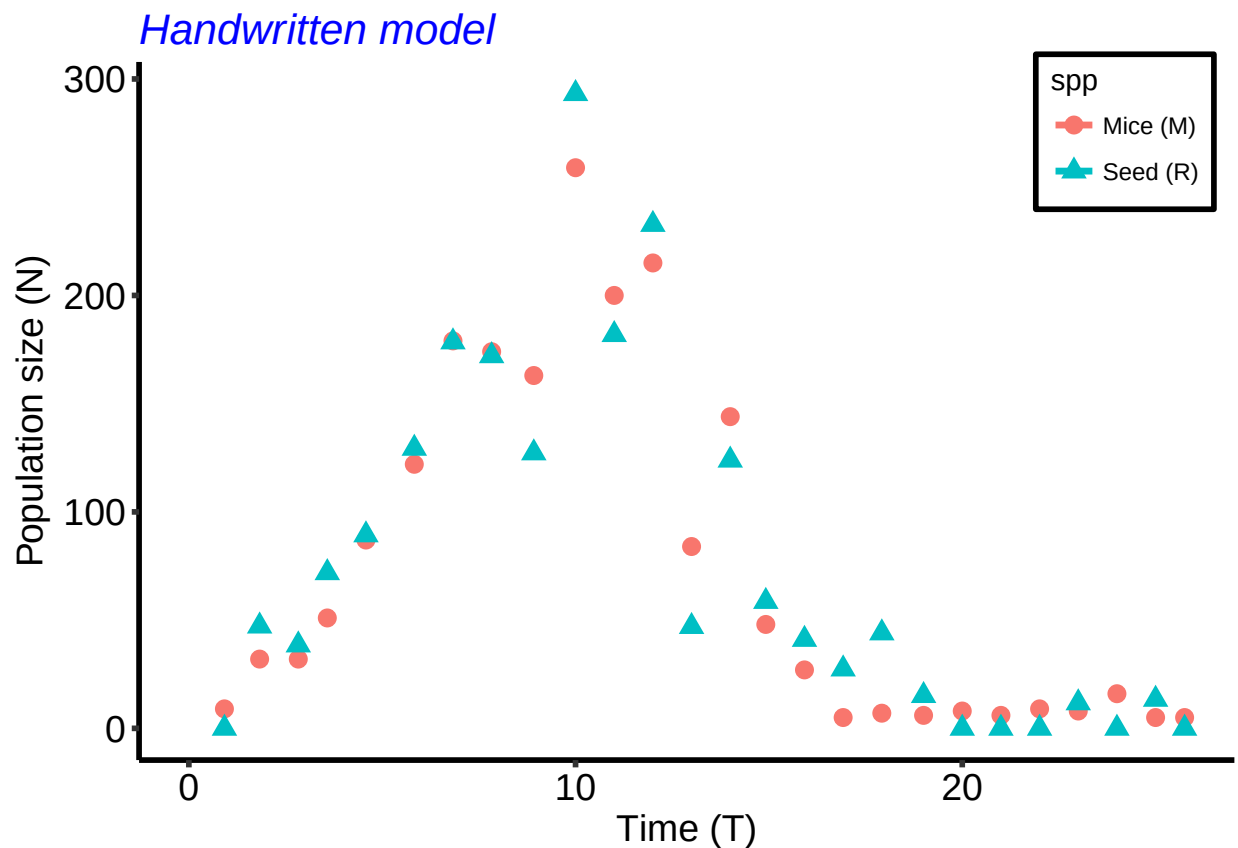
scale_colour_discrete(labels = c("Mice (M)", "Seed (R)")) +
theme_classic() +
theme(plot.title = element_text(size=16, face="italic", color="blue"),
      plot.subtitle=element_text(size=11, face="italic", color="black"),
      legend.background = element_rect(fill="white", size=1, linetype="solid", colour = "black"),
      legend.position = c(0.9,0.9),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      axis.title.y=element_text(colour = "black",size =14),
      axis.title.x = element_text(colour = "black",size =14),
      axis.text.y=element_text(colour = "black",size = 14),
      axis.text.x=element_text(colour = "black",size = 14),
      axis.ticks.x = element_line(size = 1),
      axis.ticks.y =element_line(size = 1),
      axis.line.x = element_line(size = 1),
      axis.line.y = element_line(size = 1)) +
ggtitle("Handwritten model")

```

Warning: Removed 62 rows containing missing values (geom_path).

geom_path: Each group consists of only one observation. Do you need to
adjust the group aesthetic?

Warning: Removed 12 rows containing missing values (geom_point).



```

####NOT WORKING CURRENTLY
m1 <- optim(par = c(0.001,

```

```

        0.01,
        0.01,
        0.01,
        1.8,
        1.08), fn=lik.fun, y = n_animal, yr = yr)
m1

c1 <- exp(m1$par[1])
mu1 <- exp(m1$par[2])
mu2 <- exp(m1$par[3])

xx <- seq(min(yr), max(yr), 0.01)
pars <- c(c1 = 0.001,
          mu1 = 0.01,
          mu2 = 0.01,
          h = 0.01,
          Fy = 1.8,
          max.lam = 1.08)
yini <- c(R = n_food[1], M = n_animal[1])
out <- as.data.frame(ode(func = baseMice1, y = yini, parms = pars, times = xx))

#plot
#dev.off()
plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$C ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red")
#rm is given by growth rate when resources are at maximum: R = 50

(rI * kAE * K) - rM

```

Playing with food distribution through time

```

baseMice2<- function(Time, State, Pars) {

  with(as.list(c(State, Pars)), {

    seedF <- R / perM # constant food
    conM <- c1 * R * M # mice consumption rate
    denI <- mu1 * M    # density independent term
    denD <- mu2 * M * M # density dependent term
    avalF <- h * R    # mouse independant removal of seed

    dM <- conM - denI - denD
    dF <- seedF - avalF - conM

    return(list(c(dM,dF)))
  })

  ##is this where seed goes??

}

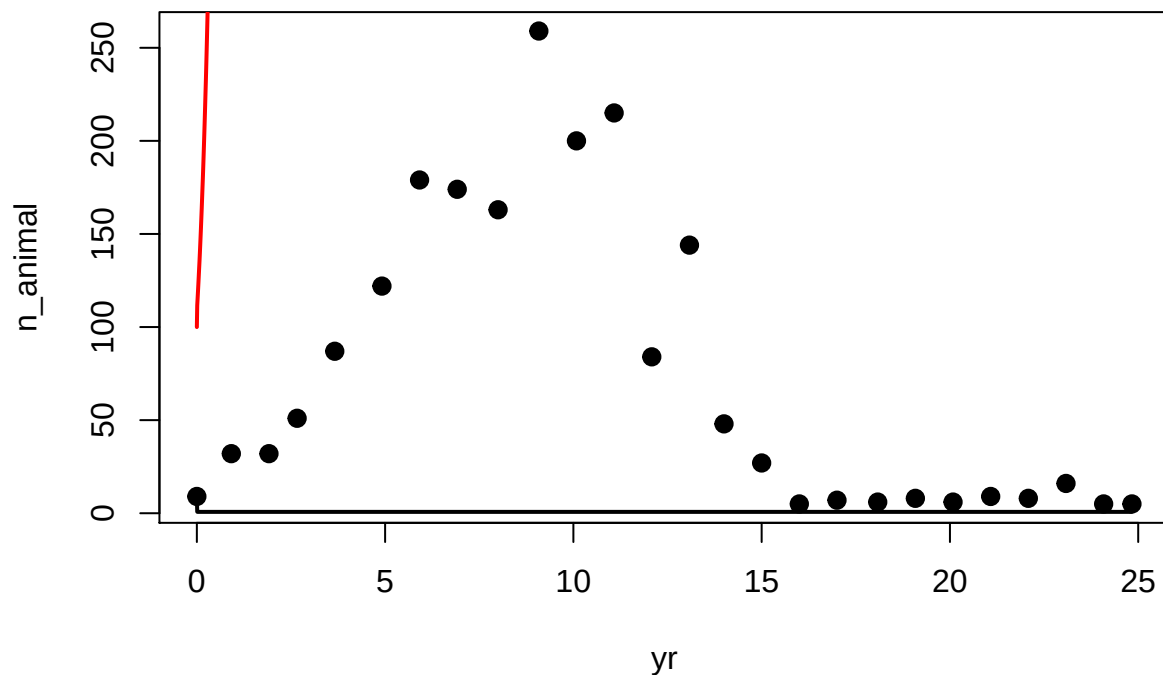
```

```

xx <- seq(min(yr), max(yr), 0.01)
pars <- c(c1 = 4.41, mu1 = -1.31, mu2 = 0.77, h = 0.81, perM = 0.25)
yini <- c(R = 100, M = n_animal[1])
out <- as.data.frame(ode(func = baseMice2, y = yini, parms = pars, times = xx))

plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$M ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red") # food....

```



#rm is given by growth rate when resources are at maximum: R = 50

```

#####STILL NOT WORKING
m1 <- optim(par = c(1, -1, -1), fn=lik.fun, y = n_animal, yr = yr)
m1

c1 <- exp(m1$par[1])
mu1 <- exp(m1$par[2])
mu2 <- exp(m1$par[3])

xx <- seq(min(yr), max(yr), 0.01)
pars <- c(c1 = c1, mu1 = mu1, mu2 = mu2)
yini <- c(R = 100, M = n_animal[1])
out <- as.data.frame(ode(func = baseMice1, y = yini, parms = pars, times = xx))

#plot
#dev.off()

```

```
plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$C ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red")
#rm is given by growth rate when resources are at maximum: R = 50

(rI * kAE * K) - rM
```

Fitted as the Holland et al. 2015 model is parameterised

Data from Holland paper

```
holland.mice <- read.csv("C:/Users/s435389/Dropbox/PhD work/Chapter 3 - MPD dataset/Holland et al2015 d
holland.seed <- read.csv("C:/Users/s435389/Dropbox/PhD work/Chapter 3 - MPD dataset/Holland et al2015 d
holland.temp <- read.csv("C:/Users/s435389/Dropbox/PhD work/Chapter 3 - MPD dataset/Holland et al2015 d
holland.meta <- read.csv("C:/Users/s435389/Dropbox/PhD work/Chapter 3 - MPD dataset/Holland et al2015 d
```

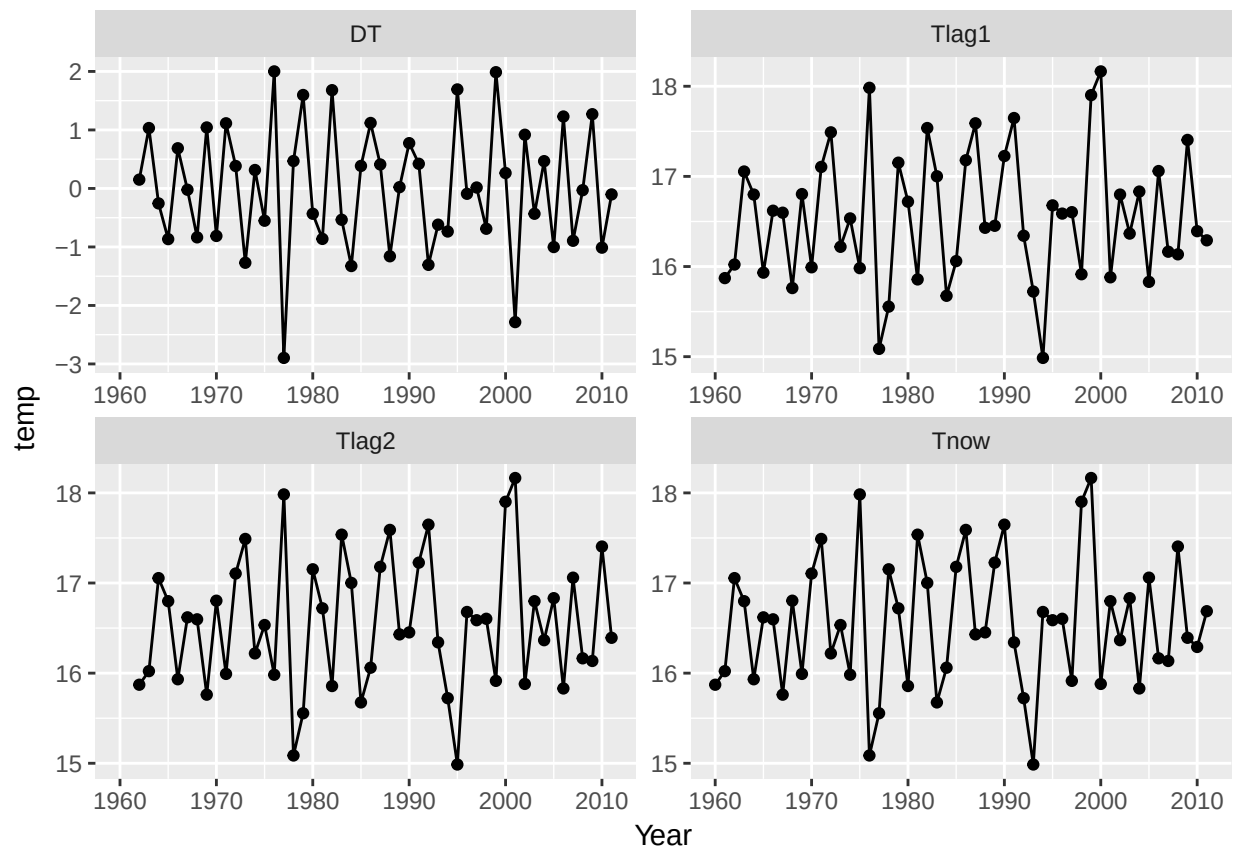
The data layout

Temperature data

Field	Description	Format	Units
Year	The year in which data were collected	Datetime	Number
Tnow	Average summer (January to March) air temperature for the year listed	Numeric	Celsius
Tlag1	Average summer (January to March) air temperature for the year prior to the year listed	Numeric	Celsius
Tlag2	Average summer (January to March) air temperature two years prior to the year listed	Numeric	Celsius
DT	Difference between Tlag1 and Tlag2	Numeric	Celsius

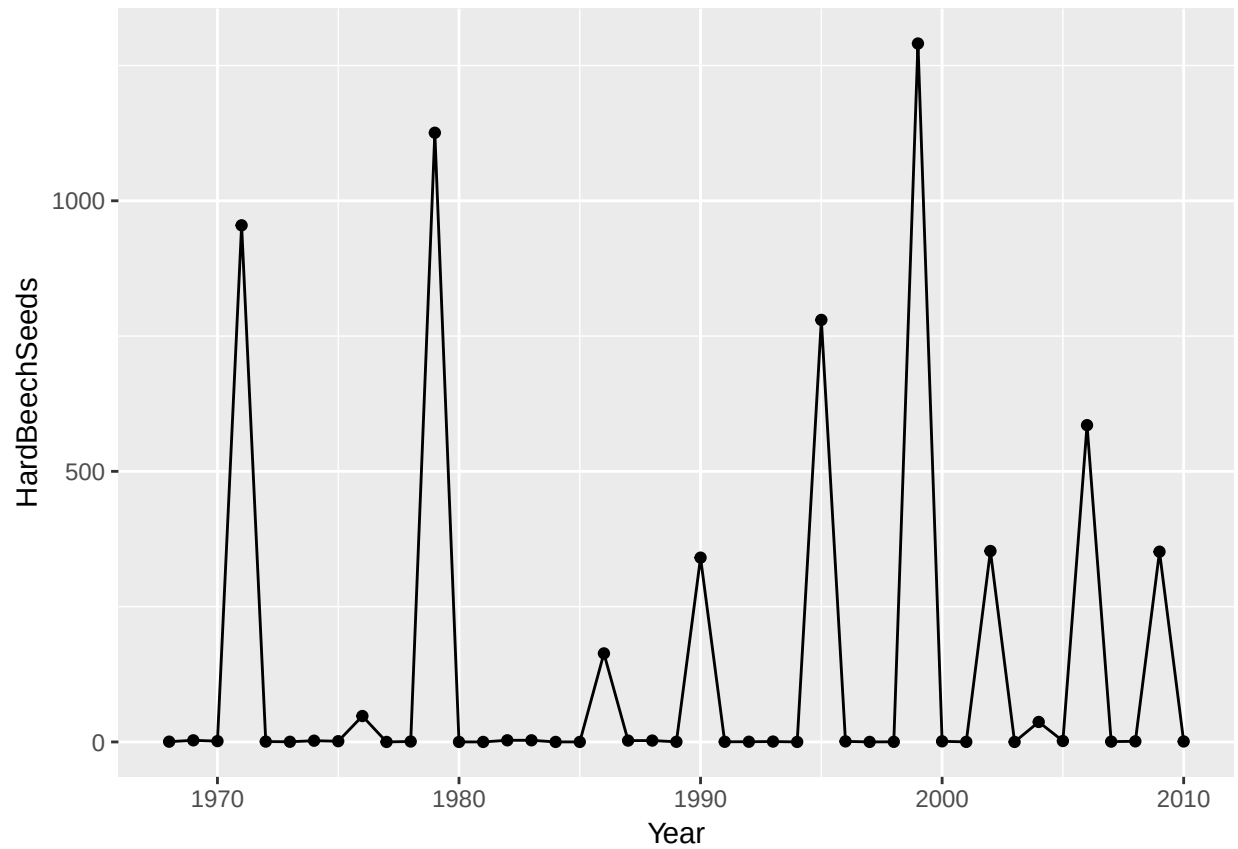
Here is a plot of the temperature over the time period

```
## Warning: Removed 5 rows containing missing values (geom_point).
## Warning: Removed 2 rows containing missing values (geom_path).
```



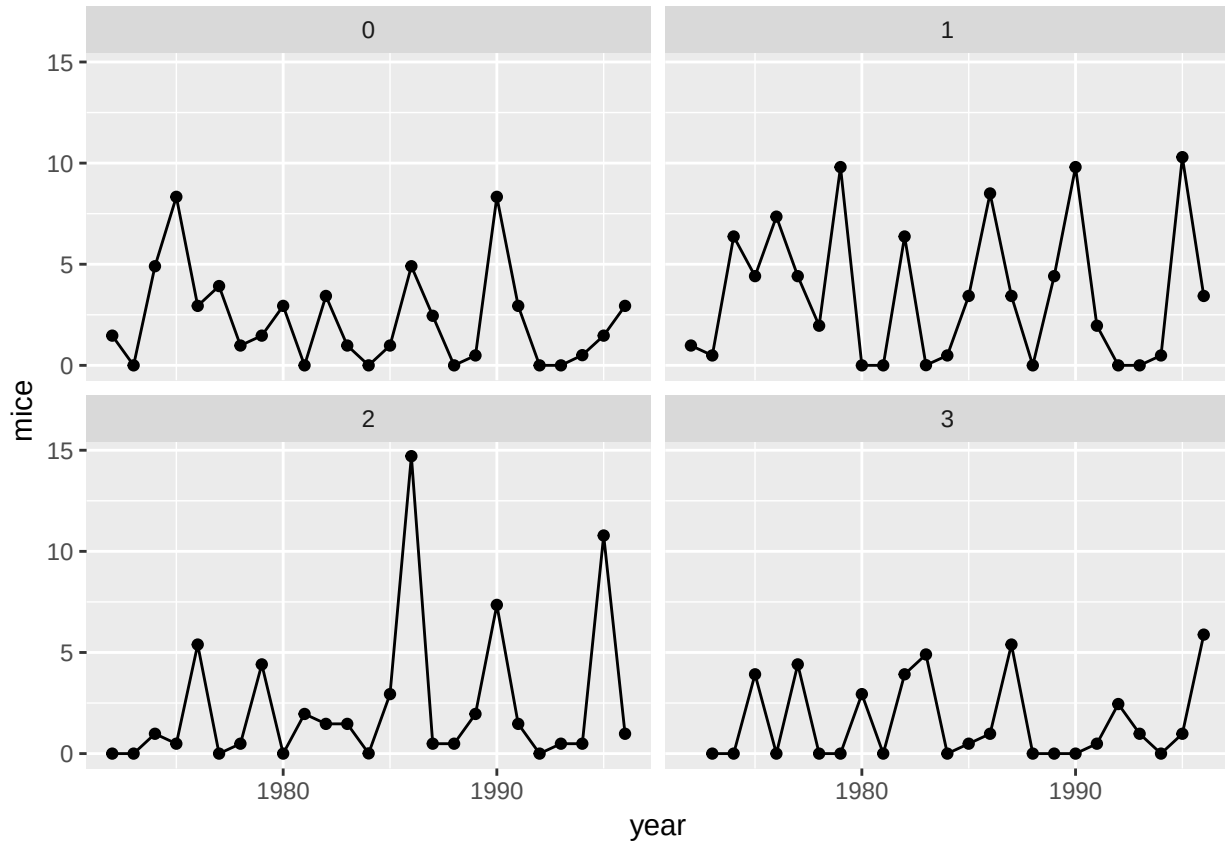
Seed data

Field	Description	Format	Units
Year	The year in which data were collected	Datetime	Number
HardBeechSeeds	Seeds produced per square metre of forest floor	Numeric	Seeds per square metre



###Mice data

Field	Description
Year	The year in which data were collected
Season	Season in which mice were captured (0=autumn; 1=winter; 2=spring; 3=summer)
Mice	Number of mice captured per 100 trap nights
Transmice	Frequency-density transformation of number of mice captured to account for trap saturation (Choquenot & Ru



model

Layout:

- consumer - resource differential equation model
- M and S are mice and seed numbers
- Here mortality increases as a function of per capita resource availability

```
# S = seed
# M = mice

#model used
Holmod0D <- function(Time, State, Pars) {

  with(as.list(c(State, Pars)), {

    #parameter structure
    GrowthM <- rI * C1 * S * K * M #growth rate of mice
    GrowthS <- S - h * S - C1 * S * K * M #seed growth
    DensIm <- rM1 * M #density independant rate
    DensDm <- rM2 * M * M #density dependant rate

    #differential equation
    dS <- GrowthS - GrowthM
    dM <- GrowthM - DenIm - DensDm
    return(list(c(dS, dM)))
  })
}
```

```

}) #states and parameters loop
} #function finished

# likelihood function
lik.fun <- function(param, y, yr) {
  rI <- exp(param[1])
  h <- exp(param[2])
  rM1 <- exp(param[3])
  rM2 <- exp(param[4])
  C1 <- exp(param[5])
  sigma <- exp(param[6])
  K <- 100
  pars <- c(rI = rI, h = h, rM1 = rM1, rM2 = rM2, C1 = C1, K = K)
  yini <- c(S = 100, M = n_animal[1])

  out <- as.data.frame(ode(func = HolmodOD, y = yini, parms = pars, times = yr))
  mu <- out$M
  logl <- sum(dnorm(y, mean = mu, sd = sigma, log = T))
  return(-logl)
}

m1 <- optim(par = c(-6.7, -1.9, -0.9, 1.9, 3.2, 2.3), fn=lik.fun, y = n_animal, yr = yr)
m1
rI <- exp(m1$par[1])
hF <- exp(m1$par[2])
rM1 <- exp(m1$par[3])
rM2 <- exp(m1$par[4])
C1 <- exp(m1$par[5])
K <- 100
xx <- seq(min(yr), max(yr), 0.01)
pars <- c(rI = rI, hF = hF, rM1 = rM1, rM2 = rM2, K = K)
yini <- c(S = 100, M = n_animal[1])
out <- as.data.frame(ode(func = HolmodOD, y = yini, parms = pars, times = xx))

```

Different versions of a two species model

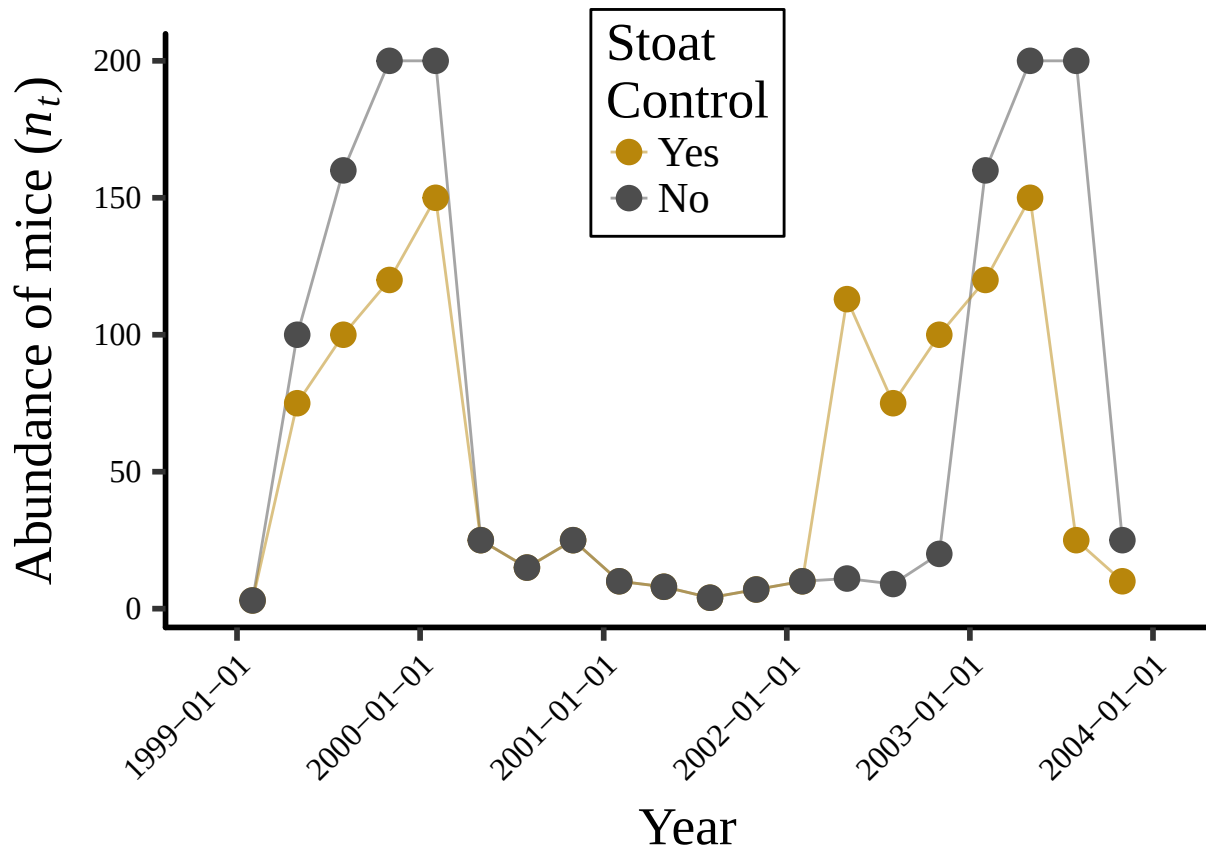
This is the more complex code taken from another book:

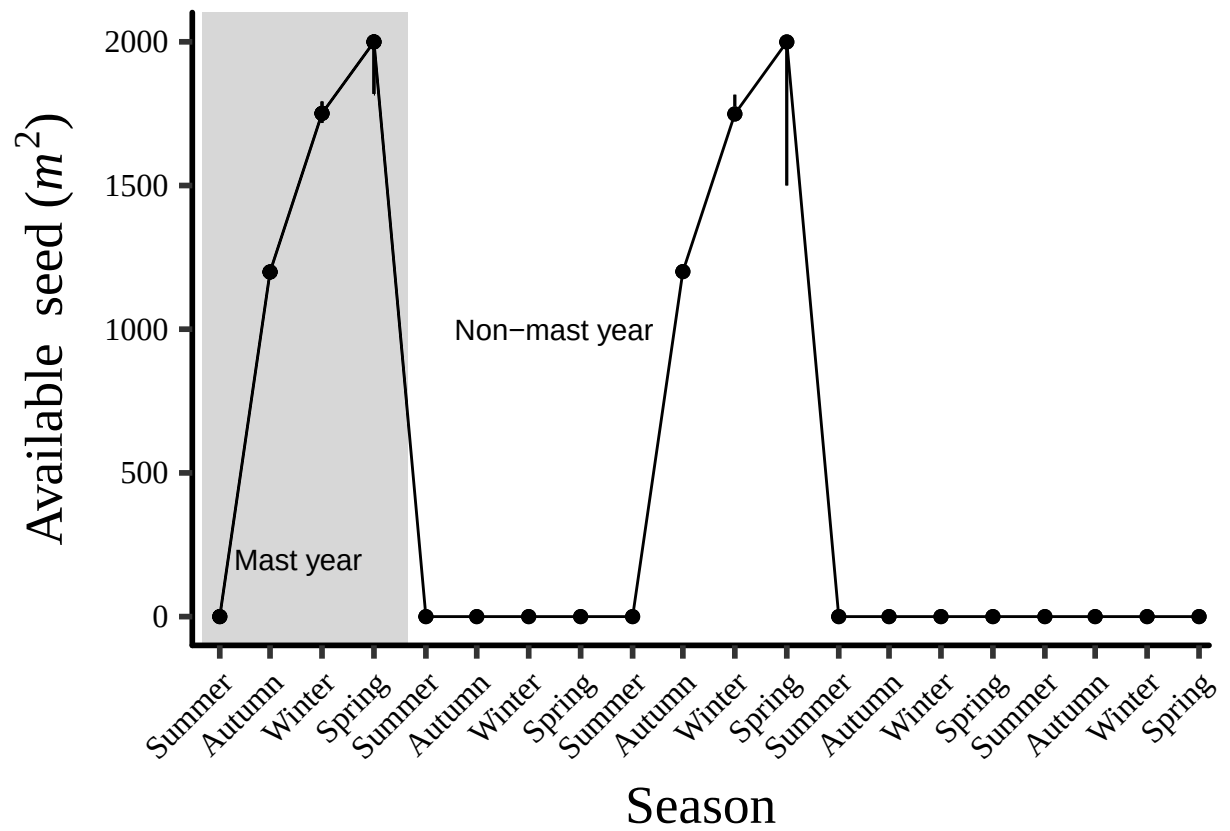
- unsure what this book is currently
- downloaded at the start of my PhD
- this stuff was based on the odesolve package (before deSolve)

Model fitted to two peaks in population size

Build a simulation for the dataset

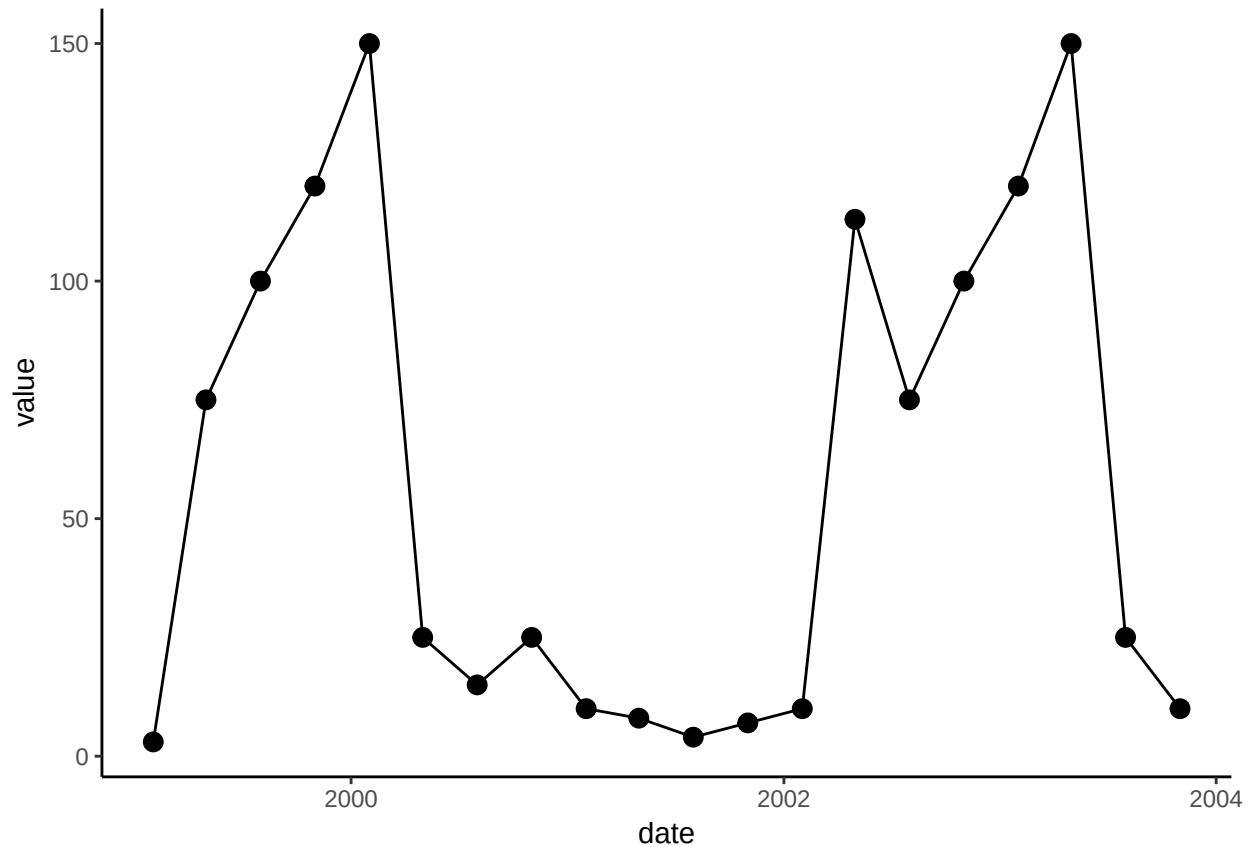
Mice





```
###seed plot
###Data used

dat1.sim <- select(dat1, value, date, control) %>%
  filter(control == "stoats")
ggplot(dat1.sim, aes(y = value, x = date)) +
  geom_point(size = 3) +
  geom_line() +
  theme_classic()
```



model

```
#####
# fit directly to N data allowing for time differences (in years) between measurements
year <- decimal_date(dat1.sim$date)
n_animal <- dat1.sim$value
# remove missing values
keep <- is.na(n_animal) == F
year <- year[keep]
n_animal <- n_animal[keep]
yr <- year - min(year)
# consumer - resource differential equation model
# C and R are consumer and resource numbers
# rI is the ingestion rate of the consumer
# rG is the growth rate of the resource
# kAE is the assimilation efficiency
# rM is the consumers mortality rate
# Here mortality increases as a function of per capita resource availability
LVmod0D <- function(Time, State, Pars) {
  with(as.list(c(State, Pars)), {
    IngestC <- rI * R * C
    GrowthR <- rG * R * (1 - R/K)
    MortC <- C * rM * (C / R)
    dR <- GrowthR - IngestC
```

```

dC <- IngestC * kAE - MortC
return(list(c(dR, dC)))
})
}
# likelihood function
lik.fun <- function(param, y, yr) {
  rI <- exp(param[1])
  rG <- exp(param[2])
  rM <- exp(param[3])
  kAE <- exp(param[4])
  sigma <- exp(param[5])
  K <- 100
  pars <- c(rI = rI, rG = rG, rM = rM, K = K, kAE = kAE)
  yini <- c(R = 100, C = n_animal[1])
  out <- as.data.frame(ode(func = LVmod0D, y = yini, parms = pars, times = yr))
  mu <- out$C
  logl <- sum(dnorm(y, mean = mu, sd = sigma, log = T))
  return(-logl)
}
m1 <- optim(par = c(-6.7, -1.9, -0.9, 1.9, 3.2), fn=lik.fun, y = n_animal, yr = yr)
m1

```

```

## $par
## [1] -4.8458929 -1.5463555 -1.4160164  0.4233515  4.0758002
##
## $value
## [1] 109.8802
##
## $counts
## function gradient
##      368      NA
##
## $convergence
## [1] 0
##
## $message
## NULL

```

```

rI <- exp(m1$par[1])
rG <- exp(m1$par[2])
rM <- exp(m1$par[3])
kAE <- exp(m1$par[4])
K <- 100
xx <- seq(min(yr), max(yr), 0.01)
pars <- c(rI = rI, rG = rG, rM = rM, kAE = kAE, K = K)
yini <- c(R = 100, C = n_animal[1])
out <- as.data.frame(ode(func = LVmod0D, y = yini, parms = pars, times = xx))

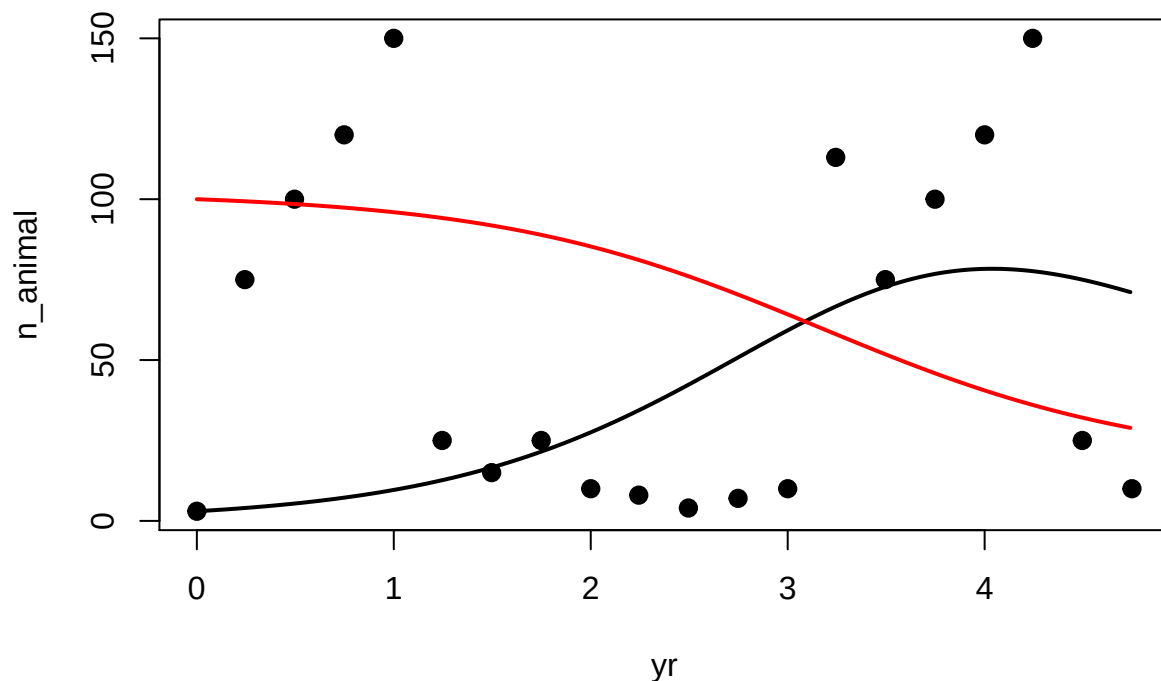
```

Results

```

plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$C ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red")

```



```
# rm is given by growth rate when resources are at maximum: R = 50
(rI * kAE * K) - rM
```

```
## [1] 0.9576899
```

trial two

With data from Holland et al. 2015 paper

```
#####
# fit directly to N data allowing for time differences (in years) between measurements
year <- decimal_date(dat$date)
n_animal <- dat$woylie
# remove missing values
keep <- is.na(n_animal) == F
```

```
## Warning in is.na(n_animal): is.na() applied to non-(list or vector) of type
## 'NULL'
```

```
year <- year[keep]
n_animal <- n_animal[keep]
yr <- year - min(year)
```

```
## Warning in min(year): no non-missing arguments to min; returning Inf
```

```
#####
```

```
#make the dataframe
```

```

#basic data

# fit directly to N data allowing for time differences (in years) between measurements
year <- decimal_date(dat1.sim$date)
n_animal <- dat1.sim$value

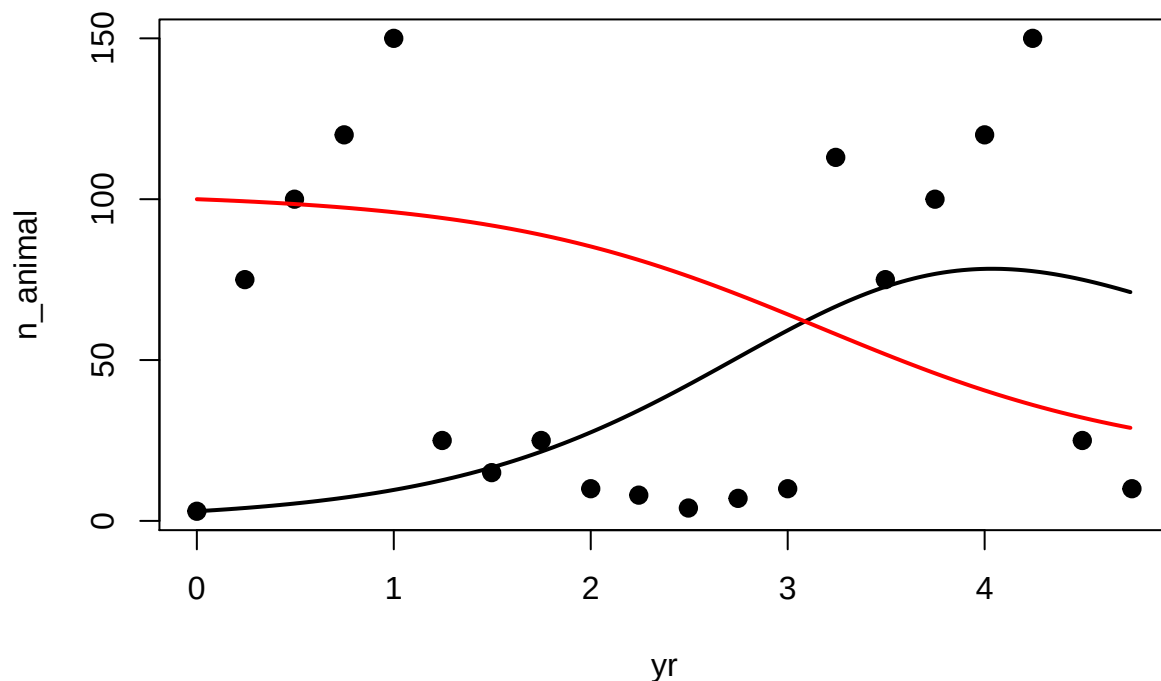
# remove missing values
keep <- is.na(n_animal) == F
year <- year[keep]
n_animal <- n_animal[keep]
yr <- year - min(year)

m1 <- optim(par = c(-6.7, -1.9, -0.9, 1.9, 3.2), fn=lik.fun, y = n_animal, yr = yr)
m1

## $par
## [1] -4.8458929 -1.5463555 -1.4160164 0.4233515 4.0758002
##
## $value
## [1] 109.8802
##
## $counts
## function gradient
##      368      NA
##
## $convergence
## [1] 0
##
## $message
## NULL

rI <- exp(m1$par[1])
rG <- exp(m1$par[2])
rM <- exp(m1$par[3])
kAE <- exp(m1$par[4])
K <- 100
xx <- seq(min(yr), max(yr), 0.01)
pars <- c(rI = rI, rG = rG, rM = rM, kAE = kAE, K = K)
yini <- c(R = 100, C = n_animal[1])
out <- as.data.frame(ode(func = LVmod0D, y = yini, parms = pars, times = xx))
plot(n_animal ~ yr, pch = 19, cex = 1.2)
lines(out$C ~ xx, lwd = 2)
lines(out$R ~ xx, lwd = 2, col = "red")

```



```
# rm is given by growth rate when resources are at maximum: R = 50
(rI * kAE * K) - rM
```

```
## [1] 0.9576899
```

Examples of basic model

This first model has very strong hunting I think

```
parms <- c(k=0.7, a=0.02, r=0.1, K=100, Th=2, d=0.25)
```

```
C <- as.numeric(a*k*Rold*Cold - d*Cold)
```

```
R <- as.numeric(r*Rold - k*Cold*Rold)
```

```
#####
# Model I:
#####
lvI <- function(t, y, p){ #exponential growth and proportional consumption:
  k=p["k"] # catch rate
  a=p["a"] # conversion efficiency from prey to predator
  d=p["d"] # mortality of predator
  r=p["r"] # growth rate of prey

  Cold = y[1]
  Rold = y[2]
```

```

C <- as.numeric(a*k*Rold*Cold - d*Cold)
R <- as.numeric(r*Rold - k*Cold*Rold)

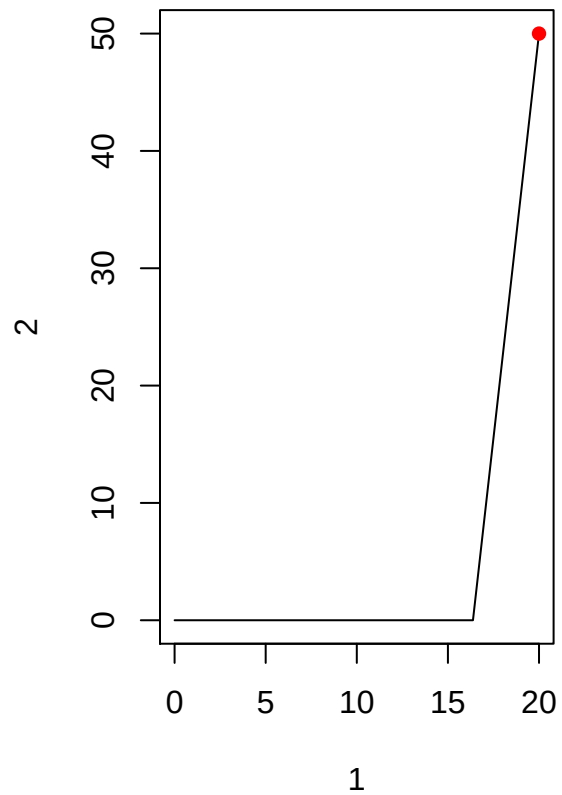
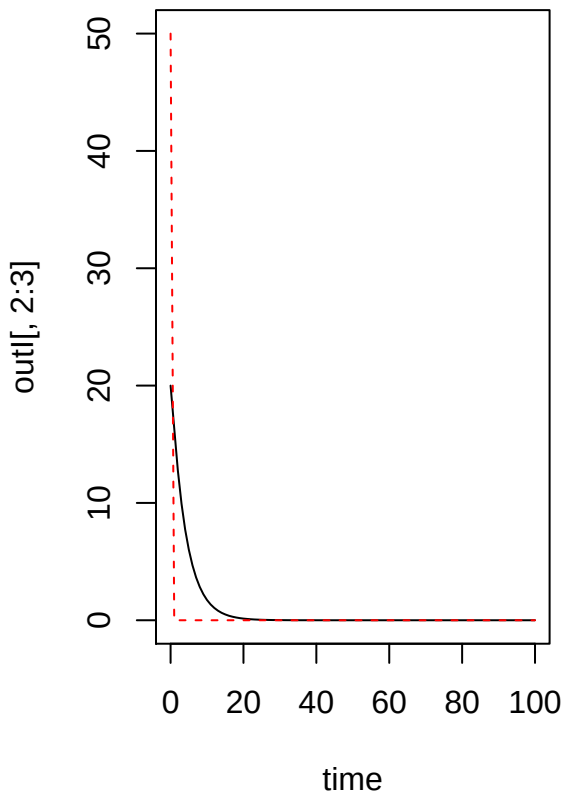
#   if (Cold<2) C = -Cold

  list(c(C=C, R=R))
}
parms <- c(k=0.7, a=0.02, r=0.1, K=100, Th=2, d=0.25)

outI <- lsoda(y=c(20, 50), times=seq(0, 100, 1), func=lvI, parms=parms)

par(mfrow=c(1,2), mar=c(5,5,1,1))
matplot(outI[,1], outI[,2:3], type="l", xlab="time",
        ylim=c(0,max(outI[,2:3])))
plot(outI[,2:3], type="l") #phase diagram
points(outI[1,2], outI[1,3], pch=16, col="red") # where it all started ...

```



```
outI[1:10,]
```

```

##      time      1      2
## [1,]  0 20.000000 5.000000e+01
## [2,]  1 16.374641 1.285752e-04
## [3,]  2 12.752479 -1.122227e-06
## [4,]  3  9.931727 -9.956450e-09
## [5,]  4  7.734840  5.201412e-09
## [6,]  5  6.023893  7.012786e-07

```

```
## [7,]      6  4.691411 -2.975044e-07
## [8,]      7  3.653678 -2.148904e-08
## [9,]      8  2.845484 -1.670809e-08
## [10,]     9  2.216064 -2.348169e-09
```

Stable model ending with one species on top

```
parms <- c(k=0.4, a=0.04, r=0.4, K=50, Th=1, d=0.2)
C <- as.numeric(a*k*Rold*Cold - d*Cold)
R <- as.numeric(r*Rold*(1-Rold/K) - k*Cold*Rold)

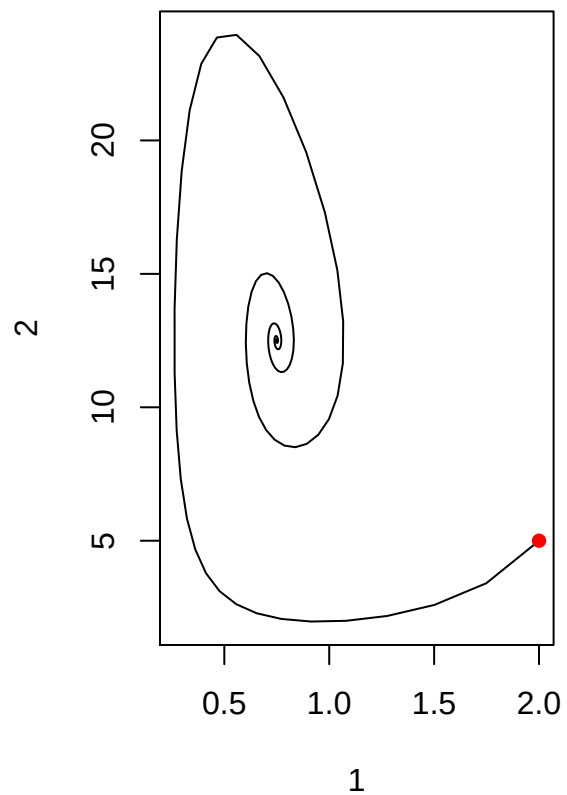
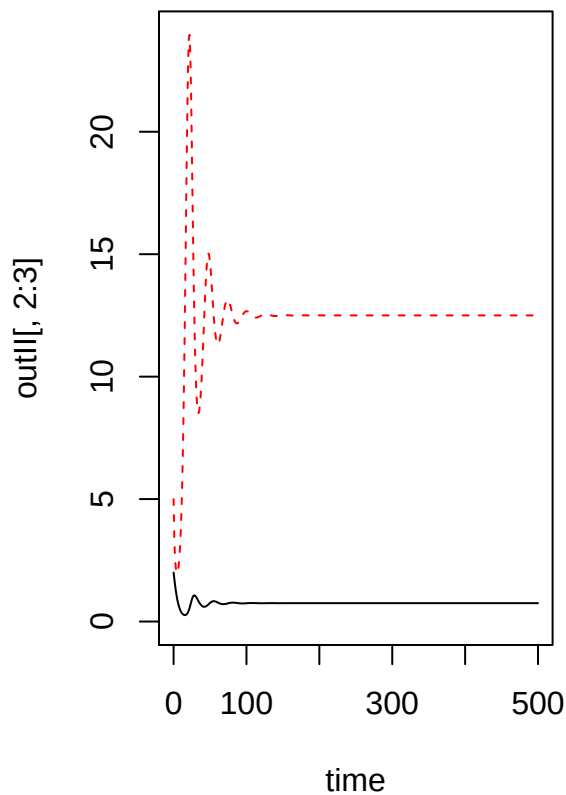
#####
# Model II:
#####
lvII <- function(t, y, p){ #logistic growth of prey and proportional consumption:
  k=p["k"] # catch rate
  a=p["a"] # conversion efficiency from prey to predator
  d=p["d"] # mortality of predator
  r=p["r"] # growth rate of prey
  K=p["K"] # carrying capacity for prey

  Cold = y[1]
  Rold = y[2]

  C <- as.numeric(a*k*Rold*Cold - d*Cold)
  R <- as.numeric(r*Rold*(1-Rold/K) - k*Cold*Rold)

  list(c(C=C, R=R))
}
parms <- c(k=0.4, a=0.04, r=0.4, K=50, Th=1, d=0.2)
outII <- lsoda(y=c(2, 5), times=seq(0, 500, 1), func=lvII, parms=parms)
par(mfrow=c(1,2), mar=c(5,5,1,1))
matplot(outII[,1], outII[,2:3], type="l", xlab="time", ylim=c(0,max(outII[,2:3])))

#base plot
plot(outII[,2:3], type="l") #phase diagram
points(outII[1,2], outII[1,3], pch=16, col="red") # where it all started ...
```



Including functional responses

Type I

```
#####
# Model II:
#####
lvII <- function(t, y, p){ #logistic growth of prey and proportional consumption:
  k=p["k"] # catch rate
  a=p["a"] # conversion efficiency from prey to predator
  d=p["d"] # mortality of predator
  r=p["r"] # growth rate of prey
  K=p["K"] # carrying capacity for prey

  Cold = y[1]
  Rold = y[2]

  C <- as.numeric(a*k*Rold*Cold - d*Cold)
  R <- as.numeric(r*Rold*(1-Rold/K) - k*Cold*Rold)

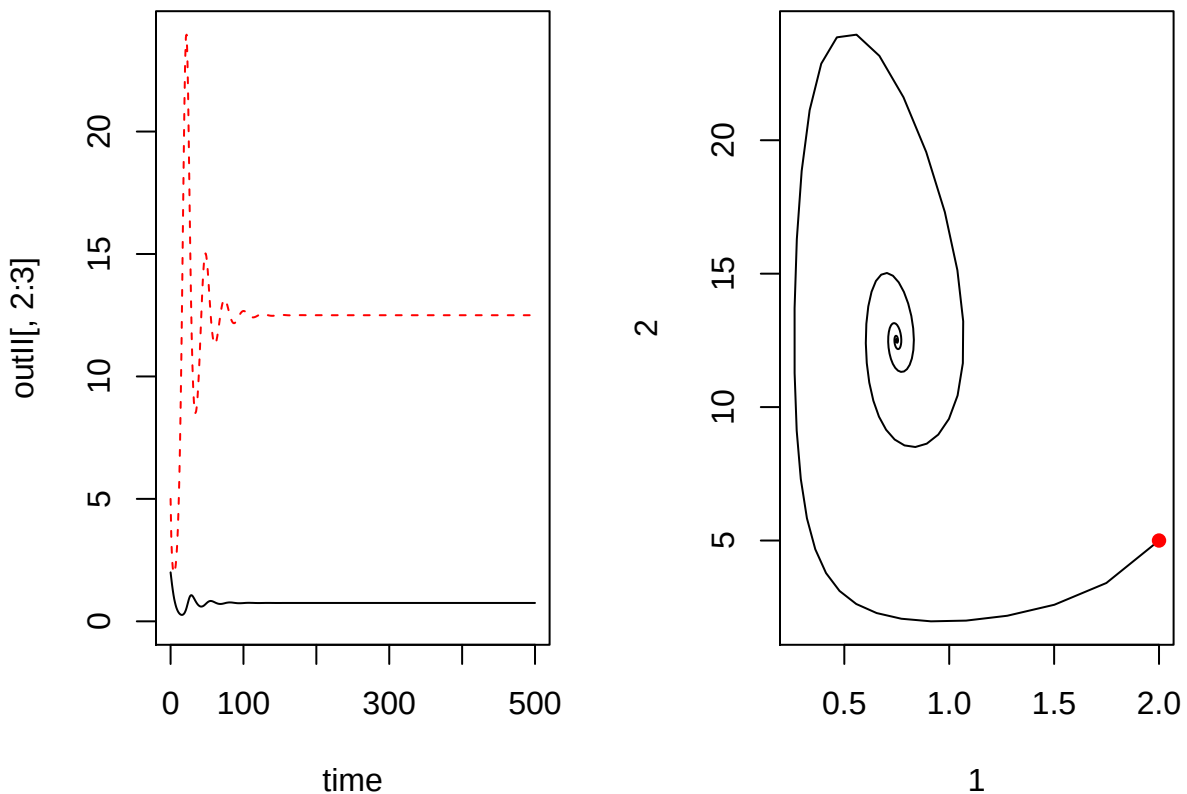
  list(c(C=C, R=R))
}
parms <- c(k=0.4, a=0.04, r=0.4, K=50, Th=1, d=0.2)
```

```

outII <- lsoda(y=c(2, 5), times=seq(0, 500, 1), func=lvII, parms=parms)
par(mfrow=c(1,2), mar=c(5,5,1,1))
matplot(outII[,1], outII[,2:3], type="l", xlab="time", ylim=c(0,max(outII[,2:3])))

#base plot
plot(outII[,2:3], type="l") #phase diagram
points(outII[1,2], outII[1,3], pch=16, col="red") # where it all started ...

```



Type II

```

#####
# Model IV:
#####
lvIV <- function(t, y, p){ # logistic growth of prey and type II functional response (p. 270 in Case)
  k=p["k"] # catch rate
  a=p["a"] # conversion efficiency from prey to predator
  d=p["d"] # mortality of predator
  r=p["r"] # growth rate of prey
  K=p["K"] # carrying capacity for prey
  Th=p["Th"] # prey handling time of predator

  Cold = y[1]
  Rold = y[2]

```

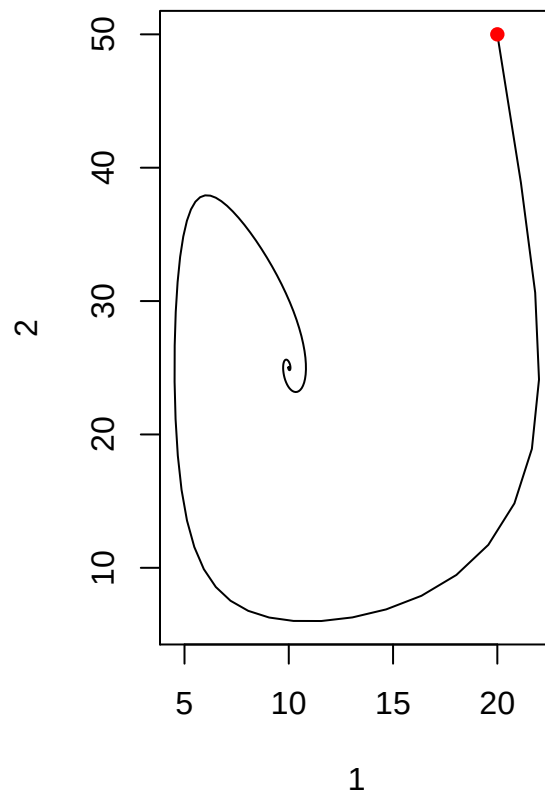
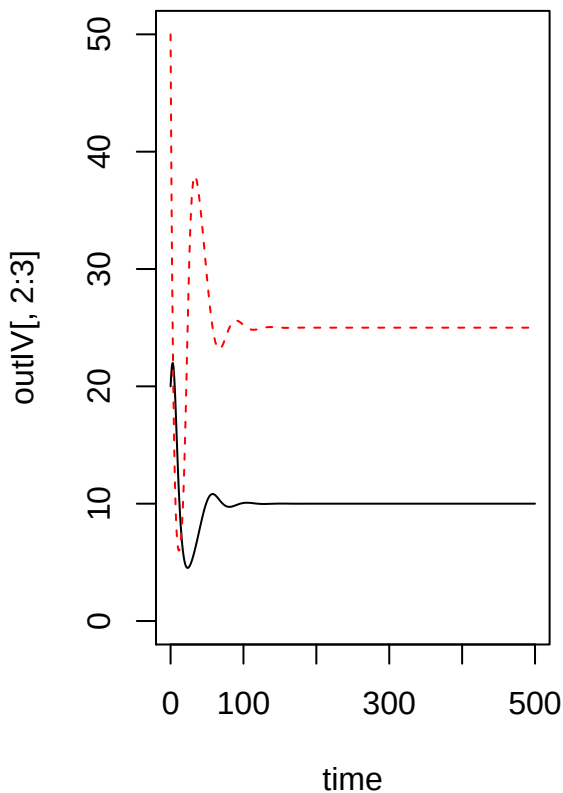
```

C <- as.numeric(k*Cold*(a*Rold / (1+a*Th*Rold)) - d*Cold)
R <- as.numeric(r*Rold*(1-Rold/K) - Cold*(a*Rold / (1+a*Th*Rold)))

list(c(C=C, R=R))
}

# functional response type III:
#parms <- c(k=0.7, a=0.02, r=0.2, K=100, Th=2, d=0.25) # 0.7 boringly stable
parms <- c(k=0.9, a=0.02, r=0.2, K=80, Th=2, d=0.25) # dampened coexistence
#parms <- c(k=0.9, a=0.02, r=0.2, K=100, Th=2, d=0.25) # stable limit cycle
#parms <- c(k=1, a=0.02, r=0.2, K=100, Th=2, d=0.25) # stable limit cycle
#parms <- c(k=.75, a=0.02, r=0.2, K=80, Th=2, d=0.35) # predator extinction
parms <- c(k=0.4, a=0.04, r=0.4, K=50, Th=1, d=0.2)
outIV <- lsoda(y=c(20, 50), times=seq(0, 500, 1), func=lvIV, parms=parms)
par(mfrow=c(1,2), mar=c(5,5,1,1))
matplot(outIV[,1], outIV[,2:3], type="l", ylim=c(0, max(outIV[,2:3])), xlab="time")
plot(outIV[,2:3], type="l") #phase diagram
points(outIV[1,2], outIV[1,3], pch=16, col="red") # where it all started ...

```



```
outIV[1:10,]
```

```

##      time      1      2
## [1,]    0 20.00000 50.000000
## [2,]    1 21.13070 38.869364
## [3,]    2 21.81487 30.624764
## [4,]    3 21.99836 24.122291

```

```
## [5,] 4 21.65703 18.919614
## [6,] 5 20.81818 14.827697
## [7,] 6 19.56784 11.721594
## [8,] 7 18.03529 9.464665
## [9,] 8 16.36164 7.901500
## [10,] 9 14.67012 6.881423
```

Two predators

```
#####
# Model V: 2 predators, one prey; predators differ in catch and death rate
#####
lvV <- function(t, y, p){ # logistic growth of prey and type II functional response (p. 270 in Case)
  k1=p["k1"] # catch rate
  k2=p["k2"] # catch rate
  a=p["a"] # conversion efficiency from prey to predator
  d1=p["d1"] # mortality of predator
  d2=p["d2"] # mortality of predator
  r=p["r"] # growth rate of prey
  K=p["K"] # carrying capacity for prey
  Th=p["Th"] # prey handling time of predator

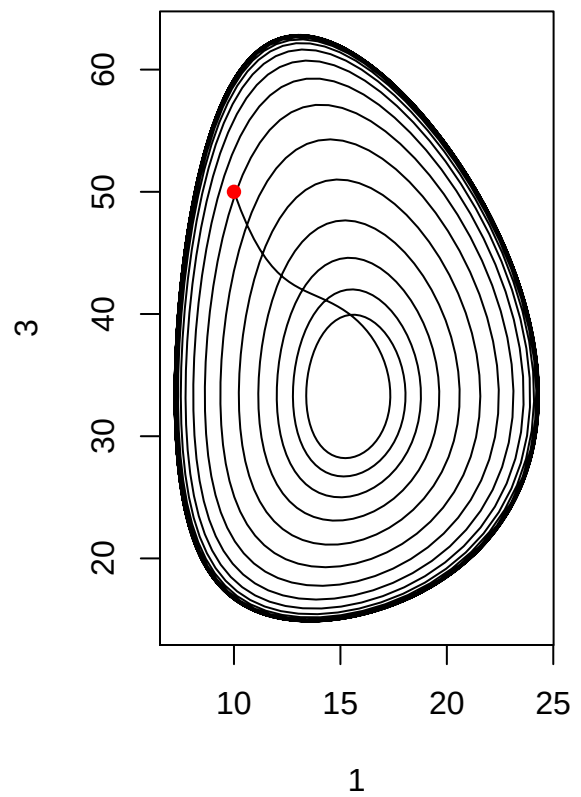
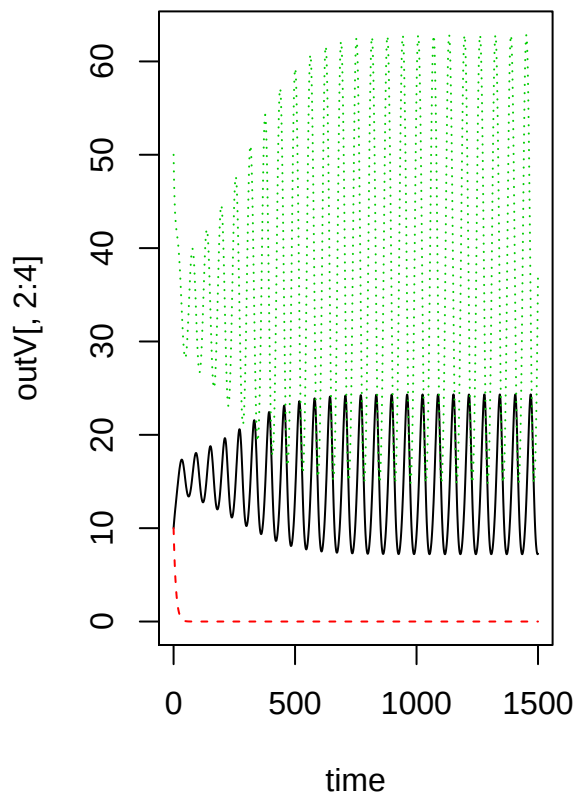
  C1old = y[1]
  C2old = y[2]
  Rold = y[3]

  C1 <- as.numeric(k1*C1old*(a*Rold / (1+a*Th*Rold)) - d1*C1old)
  C2 <- as.numeric(k2*C2old*(a*Rold / (1+a*Th*Rold)) - d2*C2old)

  R <- as.numeric(r*Rold*(1-Rold/K) -C1old*(a*Rold / (1+a*Th*Rold))
    -C2old*(a*Rold / (1+a*Th*Rold)))

  list(c(C1=C1, C2=C2, R=R))
}

# functional response type III:
parms <- c(k1=0.7, k2=0.3, a=0.02, r=0.2, K=100, Th=2, d1=0.2, d2=0.2) # 0.7 boringly stable
outV <- lsoda(y=c(10, 10, 50), times=seq(0, 1500, 1), func=lvV, parms=parms)
par(mfrow=c(1,2), mar=c(5,5,1,1))
matplot(outV[,1], outV[,2:4], type="l", ylim=c(0, max(outV[,2:4])), xlab="time")
plot(outV[,c(2,4)], type="l") #phase diagram
points(outV[1,2], outV[1,4], pch=16, col="red") #
```



```
outV[1:10,]
```

##	time	1	2	3
##	[1,]	0 10.00000	10.000000	50.00000
##	[2,]	1 10.32614	9.043569	48.47345
##	[3,]	2 10.63883	8.170688	47.19768
##	[4,]	3 10.93956	7.375872	46.13390
##	[5,]	4 11.22994	6.653586	45.24928
##	[6,]	5 11.51155	5.998355	44.51566
##	[7,]	6 11.78597	5.404847	43.90855
##	[8,]	7 12.05465	4.867939	43.40654
##	[9,]	8 12.31892	4.382760	42.99071
##	[10,]	9 12.57994	3.944723	42.64423

a time-discrete predator-prey model: Nicholson-Bailey

```
P[t] = c*H[t-1]*(1-exp(-a*P[t-1]))
H[t] = r*H[t-1]*exp(-a*P[t-1])
```

Parameters:

```
c=0.8 # survival probability of parasitoids in hosts
r=2 # host growth rate
```

```

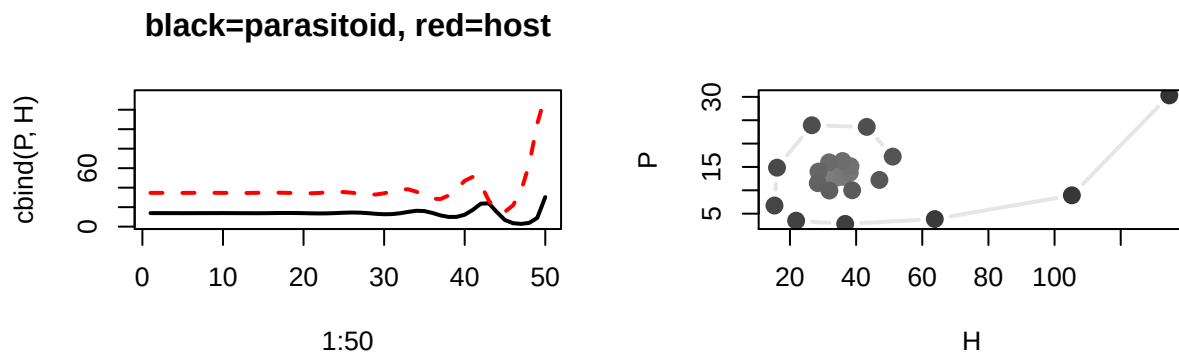
a=0.05 # search efficiency of parasitoid
# Parasitoid-Host-model:
#  $PT+1 = c*HT*(1-\exp(-a*PT))$ 
#  $HT+1 = r*HT*\exp(-a*PT)$ 

c=0.8 # survival probability of parasitoids in hosts
r=2 # host growth rate
a=0.05 # search efficiency of parasitoid

# equilibrium
P = log(r)/a + 0.01
H = r/(r-1)* log(r)/(a*c) + 0.01

for (t in 2:50){
  P[t] = c*H[t-1]*(1-exp(-a*P[t-1]))
  H[t] = r*H[t-1]*exp(-a*P[t-1])
}
par(mfrow=c(2,2))
matplot(1:50, cbind(P, H), type="l", lwd=2, main="black=parasitoid, red=host")
# phase plot (the darker the later):
plot(H,P, type="b", pch=16, col=grey(seq(0.9,0.2,len=50)), cex=1.5, lwd=2)

```



With host carrying capacity

```

P[t] = c*H[t-1]*(1-exp(-a*P[t-1]))

```

```

H[t] = H[t-1]*exp(r*(1-H[t-1]/K)-a*P[t-1])
# with carrying capacity for the host:
# PT+1 = c*HT*(1-exp(-a*PT))
# HT+1 = HT*exp(r*(1-HT/K) - a*PT)

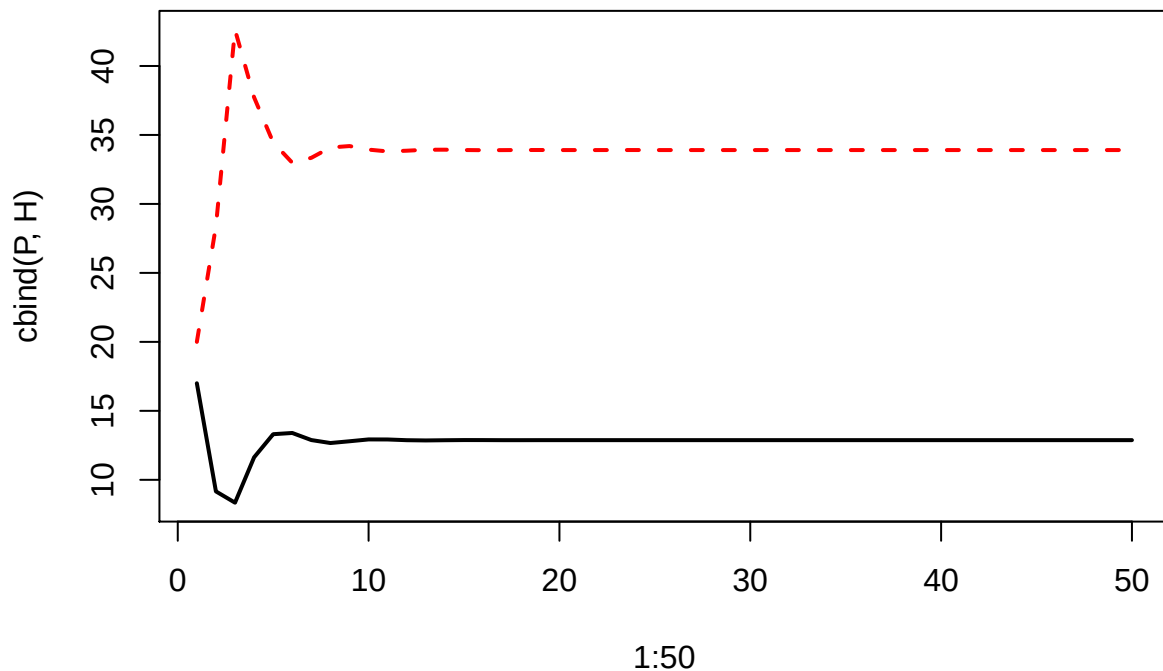
c=0.8; r=2; a=0.05; K=50#0.045

P = 17; H = 20

for (t in 2:50){
  P[t] = c*H[t-1]*(1-exp(-a*P[t-1]))
  H[t] = H[t-1]*exp(r*(1-H[t-1]/K)-a*P[t-1])
}
matplot(1:50, cbind(P, H), type="l", lwd=2, main="black=parasitoid, red=host")

```

black=parasitoid, red=host



```

# phase plot (the darker the later):
plot(H,P, type="b", pch=16, col=grey(seq(0.9,0.2,len=50)), cex=1.5, lwd=2)

```

